

With your data loaded, you now have a list of Document objects (or a list of Nodes). It's time to build an `Index` over these objects so you can start querying them.

What is an Index?

In LlamaIndex terms, an `Index` is a data structure composed of `Document` objects, designed to enable querying by an LLM. Your Index is designed to be complementary to your querying strategy.

LlamaIndex offers several different index types. We'll cover the two most common here.

Vector Store Index

A `VectorStoreIndex` is by far the most frequent type of Index you'll encounter. The Vector Store Index takes your Documents and splits them up into Nodes. It then creates `vector embeddings` of the text of every node, ready to be queried by an LLM.

What is an embedding?

`Vector embeddings` are central to how LLM applications function.

A `vector embedding`, often just called an embedding, is a **numerical representation of the semantics, or meaning of your text**. Two pieces of text with similar meanings will have mathematically similar embeddings, even if the actual text is quite different.

This mathematical relationship enables **semantic search**, where a user provides query terms and LlamaIndex can locate text that is related to the **meaning of the query terms** rather than simple keyword matching. This is a big part of how Retrieval-Augmented Generation works, and how LLMs function in general.

There are [many types of embeddings](#), and they vary in efficiency, effectiveness and computational cost. By default LlamaIndex uses `text-embedding-ada-002`, which is the default embedding used by OpenAI. If you are using different LLMs you will often want to use different embeddings.

Vector Store Index embeds your documents

Vector Store Index turns all of your text into embeddings using an API from your LLM; this is what is meant when we say it “embeds your text”. If you have a lot of text, generating embeddings can take a long time since it involves many round-trip API calls.

When you want to search your embeddings, your query is itself turned into a vector embedding, and then a mathematical operation is carried out by `VectorStoreIndex` to rank all the

embeddings by how semantically similar they are to your query.

Top K Retrieval

Once the ranking is complete, `VectorStoreIndex` returns the most-similar embeddings as their corresponding chunks of text. The number of embeddings it returns is known as `k`, so the parameter controlling how many embeddings to return is known as `top_k`. This whole type of search is often referred to as “top-k semantic retrieval” for this reason.

Top-k retrieval is the simplest form of querying a vector index; you will learn about more complex and subtler strategies when you read the [querying](#) section.

Using Vector Store Index

To use the Vector Store Index, pass it the list of Documents you created during the loading stage:

```
from llama_index.core import VectorStoreIndex

index = VectorStoreIndex.from_documents(documents)
```

Tip

`from_documents` also takes an optional argument `show_progress`. Set it to `True` to display a progress bar during index construction.

You can also choose to build an index over a list of Node objects directly:

```
from llama_index.core import VectorStoreIndex

index = VectorStoreIndex(nodes)
```

With your text indexed, it is now technically ready for [querying](#)! However, embedding all your text can be time-consuming and, if you are using a hosted LLM, it can also be expensive. To save time and money you will want to [store your embeddings](#) first.

Summary Index

A Summary Index is a simpler form of Index best suited to queries where, as the name suggests, you are trying to generate a summary of the text in your Documents. It simply stores all of the Documents and returns all of them to your query engine.

Further Reading

If your data is a set of interconnected concepts (in computer science terms, a “graph”) then you may be interested in our [knowledge graph index](#).